

Hardware-Algorithm Compatibility Report

LKA System — aBAJA SAEINDIA 2026 | Software Virtual World Simulation

1. Purpose

This report satisfies Deliverable #12 of the submission guidelines. It demonstrates that the LKA algorithm is compatible with a physically deployable embedded compute platform, meets the required real-time performance thresholds (Perception ≥ 10 Hz, Control ≥ 20 Hz), and complies with the rulebook sensor and compute budget cap.

2. Proposed Compute Platform

The target deployment platform is the NVIDIA Jetson Nano (4 GB). This platform provides sufficient compute for the BEV + Sliding Window pipeline at 30 Hz, is physically small enough to mount on the aBAJA vehicle chassis, and is commercially available at a cost that fits within the budget cap.

Component	Specification
Platform	NVIDIA Jetson Nano 4GB Developer Kit
CPU	Quad-core ARM Cortex-A57 @ 1.43 GHz
GPU	128-core Maxwell (for optional DL inference)
RAM	4 GB LPDDR4
Storage	32 GB microSD
OS	Ubuntu 20.04 (JetPack 4.6)
Power consumption	5–10 W (compatible with vehicle 12V supply)
Form factor	100 × 80 mm — mountable in vehicle electronics bay

3. Algorithm Computational Analysis

The LKA algorithm was designed to run within the compute constraints of the Jetson Nano. The key design decision — choosing BEV + Sliding Window over a deep learning approach — was driven by real-time performance requirements on embedded hardware.

3.1 Per-Frame Operation Breakdown

Pipeline Stage	Operations	Approx. Cost (Cortex-A57)
TCP frame receive	Socket read, buffer copy	~0.5 ms
Grayscale + threshold	640×480 pixel ops (OpenCV)	~0.3 ms
BEV warp (cv2.warpPerspective)	Perspective transform, bilinear interp	~1.5 ms
Sliding window search	9 windows × histogram + centroid	~1.0 ms
Polynomial fit (np.polyfit)	1st-order fit, ~200 pts per lane	~0.5 ms
Pure Pursuit + PID computation	Scalar arithmetic	<0.1 ms

DVA write (APO)	CarMaker IPC write	~0.5 ms
CSV logging	File I/O per tick	~0.5 ms
Total estimated per frame		~4.9 ms → ~204 Hz theoretical max

At 30 Hz, one control cycle budget is 33.3 ms. The estimated 4.9 ms per frame leaves a 6× headroom margin on the Cortex-A57 CPU. Even accounting for OS scheduling jitter and TCP latency variance, the 30 Hz target is achievable. In practice, the CarMaker simulation ran the pipeline at ~30 Hz on a Windows PC (Intel Core i7); Jetson Nano benchmarks are estimated based on published ARM Cortex-A57 OpenCV performance figures.

3.2 Why BEV + Sliding Window Was Chosen Over Deep Learning

A deep learning approach (e.g., UFLD, SCNN, LaneATT) was considered and would offer superior robustness to exposure variation and partial lane markings. However, it was not selected for the following reasons:

Factor	BEV + Sliding Window	Deep Learning (e.g., UFLD V2)
Inference speed on Jetson Nano	~200 Hz (CPU only)	~30–60 Hz (GPU required)
GPU dependency	None — CPU-only	Requires Maxwell GPU + TensorRT
Training data requirement	None — parameter-based	Requires CarMaker-domain fine-tuning dataset
Development time	2–3 weeks	4–8 weeks (data collection + training)
Interpretability	Full — every stage inspectable	Black-box feature representations
Robustness to exposure changes	Medium — requires correct camera settings	High — augmentation covers exposure variation
Rulebook compliance	Full — no prohibited APIs	Full — model inference is compliant
Embedded deployment risk	Low — pure Python + OpenCV	Medium — TensorRT conversion required

Decision: BEV + Sliding Window was selected for development timeline, embedded compute efficiency, and full interpretability. Deep learning remains the recommended upgrade path for improved robustness (see Section 5).

4. Budget Compliance — Itemised Cost Breakdown

All costs are indicative retail prices (INR) at time of submission. Actual team procurement costs may vary. The total must be validated against the current rulebook budget cap.

Item	Specification	Approx. Cost (INR)
Compute platform	NVIDIA Jetson Nano 4GB Developer Kit	14 000
Camera	USB camera, ≥30 fps, 640×480 capable (real-world equivalent of 2500 RSI)	500
Camera mount	Aluminium bracket, windscreen / roll cage mount	500
MicroSD storage	32 GB Class 10 (OS + code)	400
Power supply	12V → 5V DC-DC converter (vehicle supply)	600
Wiring / connectors	USB, CAN/UART harness to vehicle	800
Enclosure	ABS project box, IP44 rated	700
TOTAL		~19 500

The total estimated hardware cost of ~INR 19 500 must be verified against the current aBAJA SAEINDIA 2026 rulebook budget cap. IMU and GNSS sensors (if added for Endurance) would add approximately INR 3 000–5 000.

5. Deep Learning Upgrade Path (Future Work)

If the team proceeds to integrate a deep learning lane detector, the recommended approach is:

Step	Action
1	Collect 500–1 000 annotated frames from CarMaker runs at the corrected exposure settings (ISO 800, F/2.8, Filmic)
2	Fine-tune UFLD V2 (Ultra-Fast Lane Detection) on the collected dataset — pre-trained weights from TuSimple/CULane are publicly available
3	Export to ONNX → convert to TensorRT FP16 for Jetson Nano GPU inference
4	Benchmark inference latency on Jetson Nano (target: <33 ms per frame)
5	Replace sensor_fusion.py lane pixel output with UFLD lane polynomial output — remainder of the pipeline (Pure Pursuit + PID) is unchanged
6	Validate RMSE and ILC metrics against the BEV baseline — document improvement

6. Real-Time Performance Summary

Requirement	Threshold	This System	Status
Perception rate	≥ 10 Hz	~30 Hz (camera-limited)	✓ PASS
Control rate	≥ 20 Hz	30 Hz (locked to camera)	✓ PASS
Per-frame latency	< 100 ms	~4.9 ms estimated	✓ PASS
Steering rate limit	Physical plausibility	0.03 rad/frame at 30 Hz = 0.9 rad/s	✓ PASS
Compute platform	Real-world deployable	NVIDIA Jetson Nano — available commercially	✓ PASS